

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Istruzioni iterative

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Istruzioni Iterative

Istruzioni iterative

- Molti calcoli sono **inerentemente ripetitivi**
- La ripetizione (**iterazione**) è implementata in Java mediante le seguenti istruzioni:
 - **while**
 - **do ... while**
 - **for**

L'istruzione while

- La sintassi dell'istruzione **while** è simile a quella dell'**if**:

```
while(<espressione booleana>
{
    <istruzioni>
}
```

- La differenza è che le istruzioni nel corpo sono eseguite **finché l'espressione booleana è vera**
 - L'**espressione booleana** viene controllata all'inizio di ogni esecuzione del corpo
- Appena l'espressione booleana è falsa (eventualmente anche subito), il **ciclo termina**

Esempio: calcolare le potenze di 2 fino a 2^N

```
public class PotenzeDi2
{
    public static void main(String[] args)
    {
        // le prime potenze di 2 fino a 2^N
        final int N = Integer.parseInt(args[0]);

        int val = 1;
        int i = 0;

        while(i <= N)
        {
            System.out.println(i+" "+val);
            val *= 2;    // v = v*2;
            i++;         // i = i+1;
        }
    }
}
```

Nota che l'intero N è
definito **costante** (**final**)

Perché se N = 31 otteniamo:
31 -2147483648??

- **Esecuzione:** java PotenzeDi2 4

- 0 1
- 1 2
- 2 4
- 3 8
- 4 16

Il costrutto do ... while

- Si comporta esattamente come il **while** ma la condizione di uscita viene verificata **alla fine dell'esecuzione del corpo del ciclo** (**invece** che all'inizio):

```
public class PotenzeDi2Dowhile
{
    public static void main(String[] args)
    {
        // le prime N potenze di 2
        final int N = Integer.parseInt(args[0]);

        int val = 1;
        int i = 0;

        do
        {
            System.out.println(i+" "+val);
            val *= 2;    // v = v*2;
            i++;         // i = i+1;
        } while(i <= N);
    }
}
```

```
public class PotenzeDi2
{
    public static void main(String[] args)
    {
        // le prime potenze di 2 fino a 2^N
        final int N = Integer.parseInt(args[0]);

        int val = 1;
        int i = 0;

        while(i <= N)
        {
            System.out.println(i+" "+val);
            val *= 2;    // v = v*2;
            i++;         // i = i+1;
        }
    }
}
```

L'istruzione for

- E' un costrutto alternativo al **while** che fornisce più flessibilità nella realizzazione di **cicli**

- La **sintassi**:

```
for (<inizializzazione>; <espressione booleana>; <incremento>)  
{  
    <istruzioni>  
}
```

```
for (<inizializzazione>; <espressione booleana>; <incremento>)  
    <istruzione>;
```

- Lo schema è il seguente:
 - **Inizializza** la variabile "di controllo"
 - **Esegui il test d'uscita** sull'espressione booleana
 - **Esegui il corpo del for**
 - Alla fine di ogni ciclo **incrementa/decrementa il valore della variabile di controllo** come specificato

Esempi di istruzioni for

calcola $1+2+\dots+N$

```
int somma = 0;
for (int k = 1; k <= N; k++) somma += k;
System.out.println(somma);
```

calcola $1*2*\dots*N$

```
int prodotto = 1;
for (int k = 1; k <= N; k++) prodotto *= k;
System.out.println(prodotto);
```

stampa una tabella
di valori di funzione

```
for (int r = 0; r <= N; r++)
    System.out.println(r + " " + 2*Math.PI*r);
```

stampa i valori di un
array di stringhe

```
String[] s = new String[] { "a", "b", "c", "d" };
for (int k = 0; k < s.length; k++)
    System.out.println(k + " " + s[k]);
```

calcola l'armonica
 $H_N = 1+1/2+\dots+1/N$

```
double somma = 0.0;
for (int k = 1; k <= N; k++) somma += 1.0/k;
```


Esempi di istruzioni for annidate

Stampa 3 righe, ciascuna con 4 asterischi

```
for (int i = 1; i <= 3; i++)  
{  
    for (int j = 1; j <= 4; j++)  
        System.out.print("*");  
    System.out.println();  
}
```

Stampa 4 righe, ciascuna con 3 asterischi

```
for (int i = 1; i <= 4; i++)  
{  
    for (int j = 1; j <= 3; j++)  
        System.out.print("*");  
    System.out.println();  
}
```

Stampa 4 righe, di lunghezza 1,2,3 e 4

```
for (int i = 1; i <= 4; i++)  
{  
    for (int j = 1; j <= i; j++)  
        System.out.print("*");  
    System.out.println();  
}
```

Stampa asterischi nelle colonne pari e \$ nelle colonne dispari

```
for (int i = 1; i <= 3; i++)  
{  
    for (int j = 1; j <= 5; j++)  
        if (j % 2 == 0) System.out.print("*");  
        else System.out.print("$");  
    System.out.println();  
}
```

Stampa una scacchiera

```
for (int i = 1; i <= 3; i++)  
{  
    for (int j = 1; j <= 5; j++)  
        if ((i+j) % 2 == 0) System.out.print("*");  
        else System.out.print(" ");  
    System.out.println();  
}
```

Equivalenza dell'istruzione for con il while

```
for (<inizializzazione>; <espressione booleana>; <incremento>)  
{  
    <istruzioni>  
}
```

è equivalente a:

```
<inizializzazione>;  
while (<espressione booleana>)  
{  
    <istruzioni>  
    <incremento>;  
}
```

Incremento non unitario / non crescente / multiplo

```
for (int i=0; i<=10; i+=3)
{
}
```

L'incremento non deve essere necessariamente unitario, dipende da cosa si intende realizzare

```
for (int i=10; i>0; i--)
{
}
```

La variabile di controllo non deve essere necessariamente crescente

```
for (int k = 0, i = 0; i <= 10; i++, k += 5)
{
    // codice dell'iterazione
}
```

Le istruzioni di inizializzazione e incremento possono riferirsi a più variabili

Esercizio: for annidati

- Scrivere un **metodo** che, dato un intero N, stampi una matrice NxN il cui elemento (i, j) vale:
 - 1 se i è un divisore di j (o viceversa);
 - 0 altrimenti.

Variabili visibili solo
all'interno del ciclo for

```
public void printMatrix(final int N)
{
    for (int i = 1; i <= N; i++)
    {
        for (int j = 1; j <= N; j++)
        {
            if ((i % j == 0) || (j % i == 0))
                System.out.print("1 ");
            else
                System.out.print("0 ");
        }
        System.out.println();
    }
}
```

printMatrix(10)

```
1 1 1 1 1 1 1 1 1 1
1 1 0 1 0 1 0 1 0 1
1 0 1 0 0 1 0 0 1 0
1 1 0 1 0 0 0 1 0 0
1 0 0 0 1 0 0 0 0 1
1 1 1 0 0 1 0 0 0 0
1 0 0 0 0 0 1 0 0 0
1 1 0 1 0 0 0 1 0 0
1 0 1 0 0 0 0 0 1 0
1 1 0 0 1 0 0 0 0 1
```

Uscire da un ciclo

- Indipendentemente dal tipo di ciclo (while, do...while, for), può essere necessario **uscire dal ciclo** durante l'esecuzione del suo corpo
- Mediante l'istruzione **break**
 - Utilizzabile **solo all'interno** di un ciclo

```
public class StampaPrimeNCifre
{
    public void stampaPrimeNCifre(final String s, final int N)
    {
        int conta = 0;
        for (int k = 0; k < s.length(); k++)
        {
            char c = s.charAt(k);
            if (Character.isDigit(c))
            {
                System.out.print(c);
                conta++;
                if (conta == N) break;
            }
        }
    }

    public static void main(String[] args)
    {
        new StampaPrimeNCifre().stampaPrimeNCifre("abc3ddfed5aafwewr94423948", 3);
    }
}
```

Metodo
statico della
classe
Character:
true se è
una **cifra**

Esce dal ciclo

Uscire da cicli annidati

- L'istruzione `break` permette di uscire solo dal ciclo corrente se non si specifica un'etichetta
- Come fare nel caso in cui si voglia uscire da una sequenza di cicli annidati?
- Si può specificare un'etichetta prima di un ciclo e uscire da quel ciclo con `break <etichetta>`

```
outer:
for (int i = 0; i < h; i++)
{
    for (int j = 0; j < w; j++)
    {
        // codice qui
        // ...
        if (j == i) break outer;
    }
}
```

Esce dal for
esterno

Break vs. return

- L'istruzione `return` interrompe l'esecuzione del metodo
- L'istruzione `break` interrompe l'esecuzione di un ciclo `for`, `while` o `do...while`.

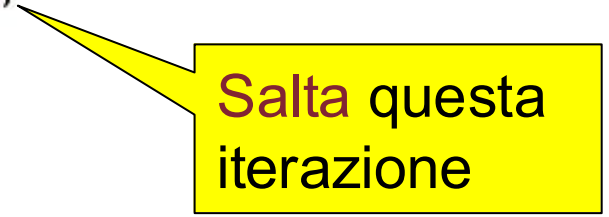
Saltare all'iterazione successiva

- Può essere utile anche saltare all'iterazione successiva

```
public class StampaPrimeNCifre2
{
    public void stampaPrimeNCifre(final String s, final int N)
    {
        int conta = 0;
        for (int k = 0; k < s.length(); k++)
        {
            char c = s.charAt(k);
            if (!Character.isDigit(c)) continue;

            System.out.print(c);
            conta++;
            if (conta == N) break;
        }
    }

    public static void main(String[] args)
    {
        new StampaPrimeNCifre2().stampaPrimeNCifre("abc3ddfed5aafwewr94423948", 3);
    }
}
```



Salta questa iterazione

Esercizio: ContaParola

- Scrivere un metodo che, presi in ingresso un testo sotto forma di stringa e una parola *w*, **trasformi il testo in parole** (token) e **conti le occorrenze di *w* nel testo**

Esercizio: StringaVerticale

- Scrivere un metodo che legge una stringa da console (ovvero da input args) e la stampa in verticale un carattere per linea
- Ad esempio, dato in input "ciao", viene stampato:

c
i
a
o

Esercizio: StringheVerticali

- Scrivere un metodo che riceve tre stringhe e le stampa in verticale una accanto all'altra
- Ad esempio: date "ciao", "buondì", "hello", stampa:

```
cbh  
iue  
aol  
onl  
do  
ì
```

Esercizio: ContaVocali

- Scrivere un metodo che riceve una stringa e stampa a video il conteggio delle vocali in essa contenute
- Ad esempio: data la stringa "le aiuole sono pulite", il metodo stampa:

a=1 e=3 i=2 o=3 u=2

Esercizio: Divisori

- Scrivere un metodo che, dato un intero positivo n in ingresso, stampi i divisori propri di n (ovvero i divisori $< n$)
- Ad esempio, dato l'intero 20, il metodo stampa:

1, 2, 4, 5, 10

Esercizio: SommaNumeriPrecedenti

- Scrivere un metodo che, dati in ingresso due interi a e b e un terzo intero N, stampi a e b e gli N numeri della sequenza in cui ogni numero è la somma dei due precedenti
- Ad esempio, dati gli interi 2, 3 e 6, il metodo stampa:

2, 3, 5, 8, 13, 21, 34, 55

Esercizio: ConversioneNumeri

- Progettare una classe per la **conversione di base dei numeri interi**
- Ogni oggetto della classe viene costruito con un **intero** o con una **stringa** che contiene **l'intero**
- La classe è in grado di convertire l'intero nella **base desiderata** (restituito sotto forma di stringa)

Esercizio: FrasePalindroma

- Una stringa è **palindroma** se rimane uguale quando viene letta da sinistra verso destra o da destra verso sinistra
- Scrivere un **metodo** che dica se una stringa è **palindroma**
- Scrivere anche una **classe di collaudo** che testi il metodo in diverse situazioni
- Ad esempio, data in ingresso le stringhe "angelalavalalegna" (o "itopinonavevanonipoti"), il metodo deve segnalare che la stringa è palindroma

Esercizio: Cornice

- Scrivere un metodo che, dato un intero N in ingresso, **stampi una cornice NxN**
- Ad esempio: dato l'intero 5 in ingresso il metodo stampa:

```
* * * * *
```

```
*       *
```

```
*       *
```

```
*       *
```

```
* * * * *
```

Esercizio: CorniceAvanzata

- Scrivere un metodo che, dato un intero N e una stringa in ingresso, stampi una cornice NxN all'interno della quale venga visualizzata la stringa (eventualmente suddivisa per righe)
- Ad esempio: dato l'intero 6 e la stringa "Cornici in Java" in ingresso il metodo stampa:

```
*****  
*Corn*  
*ici *  
*in J*  
*ava *  
*****
```

Esercizio: Terne Pitagoriche

- Una **terna pitagorica** è una tripla di numeri interi a , b , c tali che
$$1 \leq a \leq b \leq c \text{ e } a^2 + b^2 = c^2$$
 - Ovvero a e b sono i lati di un triangolo rettangolo e c l'ipotenusa
- Scrivere un metodo che legge un intero N e **stampa tutte le triple pitagoriche** con $c \leq N$
- Ad esempio: dato $N=15$ il metodo stampa:

$a=3$ $b=4$ $c=5$

$a=6$ $b=8$ $c=10$

$a=5$ $b=12$ $c=13$

$a=9$ $b=12$ $c=15$

Esercizio: StampaTriangoli

- Scrivere un metodo che, dato un intero positivo dispari $N > 1$, stampi un triangolo isoscele la cui base è costituita da N caratteri e l'altezza da $N-2$
- Ad esempio, dato l'intero 5, il metodo stampa:

*

Esercizio: la classe RettangoloDiCaratteri

- Progettare una classe `RettangoloDiCaratteri` che rappresenta un rettangolo riempito con caratteri *
- Un oggetto della classe viene costruito fornendo la posizione `x`, `y` e la lunghezza e altezza del rettangolo
- Il metodo `draw` si occupa di stampare il rettangolo a video, partendo dalla posizione (0,0)
- Ad esempio, dato il rettangolo (2, 2, 4, 3), il metodo `draw()` stampa (_ rappresenta una spazio):

```
_
_
_  * * * *
_  * * * *
_  * * * *
```

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli.

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.